MESAFARTAI – VERSÃO 2 (AC-4 parte 2 – valor: 0,5 ponto)

Versão final – arquitetura da solução:

MESAFARTAI/

├— requirements.txt

├— src/ (app.py, auth.py, chatbot.py, database.py, entities.py, geocoding.py, matchmaking.py, nlu.py)

├— data/ (alimentos.csv, dataset_intencoes.csv, dataset_teste.csv)

├— scripts/ (gerar_hash_senha.py, gerar_dataset.py)

├— .streamlit/ (secrets.toml.example)

Passo 1 – Clonar ou atualizar seu projeto no terminal do VSCODE

Abra o VSCODE e abra um novo terminal shell

- Ser for só atualização, fazer o git pull:

cd PROJETO-MESAFARTAI

git status

git pull origin main

- Se for clonar o projeto inteiro:

Git clone https://github.com/"nome_do_seu_usuario"/PROJETO-MESAFARTAI

cd PROJETO-MESAFARTAI

A diferença é:

- git clone: baixa o projeto pela primeira vez.
- git pull: atualiza um projeto que já existe na máquina.
- git push: envia suas alterações para o GitHub.

Passo 2 – Abra todo o projeto no VSCODE e crie o ambiente virtual Python

No terminal Shell, digite:

code .

python -m venv venv

Ative o ambiente:

- **Windows:** .venv\Scripts\activate
- **Mac/Linux:** source .venv/bin/activate

Atualize o pip: python.exe -m pip install --upgrade pip

Passo 3 – Atualizações e criação de novos arquivos:

1 - Atualizar requirements.txt

- Abra o arquivo requirements.txt pelo VSCODE

- Vá até o repositório do professor da aula de hoje e encontre o requirements.txt

- Copie os dados de lá para o seu arquivo requirements.txt e salve (control+s)

- Execute o comando: pip install -r requirements.txt

2 – Criar diretório scripts e o .streamlit

- Crie o diretório scripts com o comando: mkdir scripts

- Crie o diretório .streamlit com o comando: mkdir .streamlit

- Confirme se ambos foram criados com sucesso: ls -v

3 - Crie o autenticador, o secrets e a senha do administrador do sistema

- Vá até o repositório do professor da aula e encontre o arquivo .streamlit/secrets.toml

- Neste caso, baixe o arquivo e salve dentro do diretório .streamlit que acabou de criar

- Volte ao repositório do professor e encontre o arquivo scripts/gerar_hash_senha.py

- Copie os dados de lá, crie o arquivo gerar_hash_senha.py dentro do diretório scripts, cole os dados e salve (control+s)

- Volte ao repositório do professor e encontre o arquivo src/auth.py

- Copie os dados de lá, crie o arquivo auth.py dentro do diretório src, cole os dados e salve (control+s)

- Agora rode o comando para gerar a senha de administrador do sistema:

python scripts/gerar_hash_senha.py

- Siga as orientações que serão dadas na linha de comando para gerar a senha

- Copie a senha gerada no trecho ‘senha_hash =’ do arquivo .streamlit/secrets.toml

4 - Abra o .streamlit/secrets.toml e faça duas edições:

1. Substitua a linha senha_hash pelo bloco gerado no passo 3 (caso não tenha feito).
2. Em nominatim_user_agent, troque o “seu-email@exemplo.com “pelo seu e-mail g gmail real (o serviço de mapas exige essa identificação).

5 – Atualize os arquivos da pasta src na sequência abaixo:

1 - Volte ao repositório do professor e encontre o arquivo src/geocoding.py

- Copie os dados de lá, crie o arquivo geocoding.py dentro do diretório src, cole os dados e salve (control+s)

2 - Volte ao repositório do professor e encontre o arquivo src/chatbot.py

- Copie os dados de lá, crie o arquivo chatbot.py dentro do diretório src, cole os dados e salve (control+s)

3 - Volte ao repositório do professor e encontre o arquivo src/database.py

- Copie os dados de lá no arquivo database.py dentro do diretório src, cole os dados e salve (control+s)

4 - Volte ao repositório do professor e encontre o arquivo src/entities.py

- Copie os dados de lá no arquivo entities.py dentro do diretório src, cole os dados e salve (control+s)

5 - Volte ao repositório do professor e encontre o arquivo src/matchmaking.py

- Copie os dados de lá no arquivo matchmaking.py dentro do diretório src, cole os dados e salve (control+s)

6 - Volte ao repositório do professor e encontre o arquivo src/nlu.py

- Copie os dados de lá no arquivo nlu.py dentro do diretório src, cole os dados e salve (control+s)

7 - Volte ao repositório do professor e encontre o arquivo src/app.py

- Copie os dados de lá no arquivo app.py dentro do diretório src, cole os dados e salve (control+s)

6 – Criar diretório data e inserir os arquivos de dados obrigatórios

- Crie o diretório data com o comando: mkdir data

- Confirme se criado com sucesso: ls -v

- Volte ao repositório do professor e encontre os arquivos: alimentos.csv, dataset_intencoes.csv, dataset_teste.csv

7 – Rodar o aplicativo:

streamlit run src/app.py

para cancelar a aplicação basta dar CONTROL+C no terminal

8 - Testar o aplicativo e criar mais dados

1 – Entre na área de administrador e teste a senha de acesso.

2 – Crie 4 novas ONGs para receber alimentos.

3 – Façam 5 novas interações com o chatbot para informar doação dos alimentos

9 – Atualizar a versão final para o repositório

- Atualize seu repositório com a nova versão completa do sistema com os comandos abaixo no terminal, dentro da pasta raiz do projeto (PROJETO-MESAFARTAI):

git pull origin main –rebase

git status

git add .

git commit -m "MESAFARTAI: versão 2”

git push origin main

10 – Entregas de hoje (valor: 0,25)

– Coletem com prints todas as interações feitas no aplicativo

- Encaminhem por email todos os prints como evidência para a nota de hoje, pelo email::

flavio.santarelli@pro.fecaf.com.br

- No email coloquem todos os integrantes do grupo PRESENTES na aula de hoje.